Taller Práctico GIT

Introducción

Este practico te permitirá profundizar en los conceptos y comandos de Git, explorando escenarios más complejos y prácticas avanzadas para gestionar tu código de manera eficiente.

Objetivos:

- Repasar los conceptos básicos de Git (repositorio, commit, rama, etc.)
- Configurar un repositorio remoto y realizar push/pull de cambios
- Trabajar con múltiples ramas y estrategias de branching
- Resolver conflictos de merge
- Utilizar herramientas de Git para mejorar la productividad
- Implementar flujos de trabajo de Git comunes

Herramientas necesarias:

- Un editor de texto (como Visual Studio Code, Notepad++, Sublime Text, etc.)
- Terminal o línea de comandos
- Una cuenta en un servicio de alojamiento de repositorios Git (opcional) - Opciones populares: GitHub, GitLab

Herramientas de Git:

Visualizadores de Git:

- **Sourcetree:** Interfaz gráfica de usuario para visualizar y gestionar tu repositorio.
- **GitKraken:** Potente herramienta visual con funcionalidades avanzadas.

Extensiones de terminal*:*

- **Oh My Zsh:** Framework de configuración de terminal con plugins para Git.
- **PowerShell ISE:** Entorno de desarrollo integrado para PowerShell con plugins para Git.

6. Flujos de trabajo de Git:

Git Flow:

- **master:** Rama principal con el código estable.
- **develop:** Rama de desarrollo principal.
- **feature/<nombre>:** Ramas para funcionalidades nuevas.
- **hotfix/<nombre>:** Ramas para correcciones de errores urgentes.

GitHub Flow:

- **master:** Rama principal con el código estable.
- **feature/<nombre>:** Ramas para funcionalidades nuevas.
- **bugfix/<nombre>:** Ramas para correcciones de errores.

Elige el flujo de trabajo que mejor se adapte a tu proyecto y equipo.

Archivos MD: Markdown

Markdown es un lenguaje de marcado ligero creado para facilitar la escritura y el formateo de texto simple. A diferencia de HTML, que utiliza etiquetas complejas, Markdown utiliza una sintaxis intuitiva basada en caracteres comunes para definir la estructura y el estilo del texto.

Características principales de los archivos MD:

- **Formato de texto simple:** Se escribe como texto plano, sin necesidad de conocer código HTML.
- **Sintaxis intuitiva:** Utiliza caracteres comunes como asteriscos, guiones bajos y almohadillas para definir elementos como encabezados, listas y enlaces.
- **Fácil de leer y escribir:** Es más legible que el HTML y más fácil de escribir a mano.
- **Versátil:** Se puede usar para crear documentos, presentaciones, correos electrónicos y más.
- **Ampliamente compatible:** Soportado por numerosos editores de texto, plataformas de publicación y herramientas de desarrollo.

Editores de archivos MD:

Existen editores de texto que soportan Markdown, como Sublime Text, Atom, Visual Studio Code y Typora. También hay plataformas de publicación en línea que permiten crear y editar documentos Markdown, como GitHub y GitLab.

Ejemplo para la edición del archivo Markdown

Encabezados

Markdown utiliza almohadillas (#) para definir diferentes niveles de encabezados:

Encabezado de nivel 1
Encabezado de nivel 2
Encabezado de nivel 3
Encabezado de nivel 4
Encabezado de nivel 5
Encabezado de nivel 6

Párrafos

Los párrafos se escriben simplemente como líneas de texto.

Negrita:

Dos asteriscos (**) o dos guiones bajos (__) alrededor del texto.
* * Texto en Negrita**

Cursiva:

Un asterisco (*) o un guión bajo (_) alrededor del texto.

Preformateado:

Cuatro espacios al principio de cada línea.

Listas

Las listas se pueden crear con guiones bajos (**-**), asteriscos (*******) o números (***1.***):
Ejemplo:
* Introducción
* Estructuras y funciones
* Contenedores y Errores

Enlaces

Para crear un enlace, escriba el texto del enlace entre corchetes ([]) y la URL entre paréntesis (()):

[Archivo](https://github.com/...../script.js)
[archivo](otro.md)

Lista y enelace

* [1 introduccion] (carpeta/Resumen.md)
* [2. Estructuras y funciones](carperta/Resumen.md)
* [3. Contenedores y Errores](carpeta/Resumen.md)

Trabajo Practico 1.

1. Cree un Repositorio remoto en GitHub nombre TP1 – GIT
2. Inicie un repositorio local
3. Cree un archivo Readme.md con el título TP GIT y una lista de nombre apellido de los integrantes del grupo
4. Añada el archivo al staging y realice el commit.
5. Agregue el repositorio remoto.
6. Realice el push al repositorio remoto
7. Cree una rama para cada integrante del grupo cada integrante
8. Realice los Push y Pull correspondientes
9. Cada integrante deberá inicializar un repositorio y conectar con el remoto
10. Luego parado sobre su rama realizara un breve CV nombre_apellido.md
11. Realizara el link en el Archivo Readme.md, preparara los comabios, realizara el commit, push.
12. Cree una rama de feature_prueba para una funcionalidad nueva
13. En la rama agregue un archivo script.js
14. Modifique el archivo readme.md para que posea el link al archivo script.js
15. Fusione la rama master con la rama feature_prueba
16. Restaure en la rama master el archivo Readme.md a la versión anterior al Merge entre el feature_prueba , realice los commit y push.

Al finalizar se realizará un push donde el archivo Readme.md deberá poseer un índice linkeado a cada uno de los CV de los integrantes del grupo y cada integrante poseerá su rama de feature.